

Tecnología Digital VI

Ensamblés I

Licenciatura en Tecnología Digital - UTDT

Hasta ahora – Lineales vs Árboles

Regresión lineal

$$f(X) = \beta_0 + \sum_{j=1}^p X_j \beta_j,$$

Árbol de decisión

$$f(X) = \sum_{m=1}^M c_m \cdot 1_{(X \in R_m)}$$

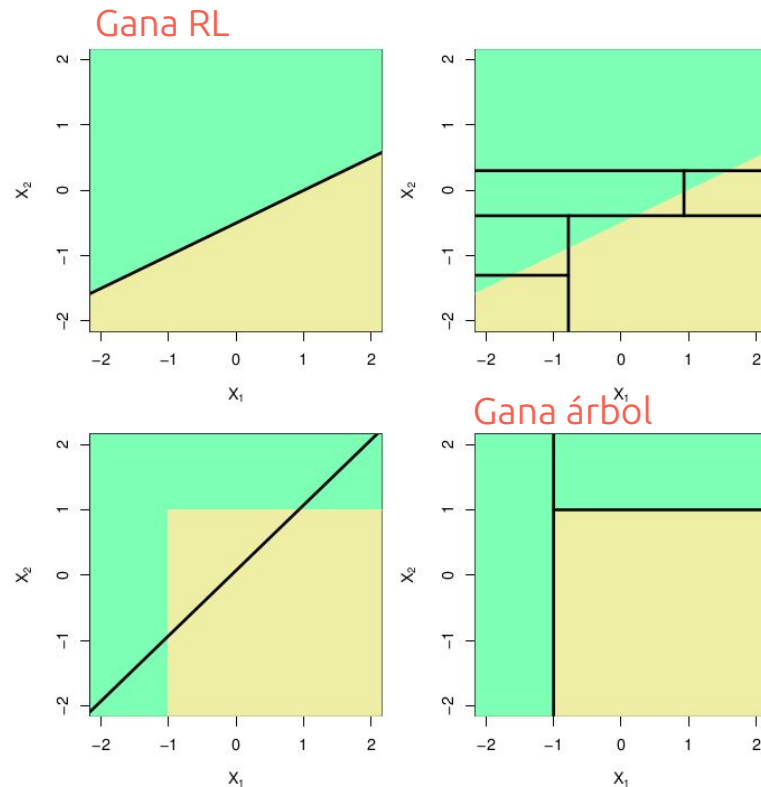
A priori son dos modelos que “suponen” hipótesis muy distintas de los datos subyacentes.

¿Cuándo puede funcionar uno más que el otro?

Decision Trees vs Modelo Lineal

Entender los modelos subyacentes nos ayudan a entender los casos ideales de funcionamiento de cada modelo y las limitaciones con las que nos vamos a encontrar.

¿Qué modelo se adapta mejor al caso en el que “pierde”?



Benchmark

Arrancamos con una
regresión lineal como
benchmark.

Considerando que los
valores están entre 6 y 43, un
MSE de 9 no parece muy
satisfactorio.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# Load the dataset into a Pandas DataFrame
df = pd.read_excel('./ENB2012_data.xlsx')

# Prepare the data
X = df.drop(['Y1', 'Y2'], axis=1) # Features
y = df['Y1'] # Target variable

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Create and train the Linear Regression model
regressor = LinearRegression()
regressor.fit(X_train, y_train)

# Make predictions on the test set
y_pred = regressor.predict(X_test)

# Calculate the mean squared error
mse = mean_squared_error(y_test, y_pred)
print("Mean Squared Error:", mse)
```

Mean Squared Error: 9.15320764594167

Otro benchmark

Otro benchmark posible es utilizar el algoritmo de k vecinos más cercanos para regresiones. En vez de “votar”, aportan un valor para luego promediar.

Mejora nuestro MSE pero todavía parece alto.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import mean_squared_error

# Load the dataset into a Pandas DataFrame
df = pd.read_excel('./ENB2012_data.xlsx')

# Prepare the data
X = df.drop(['Y1', 'Y2'], axis=1) # Features
y = df['Y1'] # Target variable

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Create and train the K-Nearest Neighbors Regressor
regressor = KNeighborsRegressor()
regressor.fit(X_train, y_train)

# Make predictions on the test set
y_pred = regressor.predict(X_test)

# Calculate the mean squared error
mse = mean_squared_error(y_test, y_pred)
print("Mean Squared Error:", mse)
```

Mean Squared Error: 4.85465490909091

Ensamblés

Un ensamble o *ensemble* es, dicho coloquialmente, un **rejunte de modelos**.

La definición aplica de manera general a **mezclar cualquier tipo de modelos** de Machine Learning para **generar uno nuevo**, pero particularmente **se utilizan mucho los árboles de decisión como modelo subyacente**.

En un ensamble hay varias preguntas a responder:

- ¿**Qué modelos subyacentes usamos?** ¿**Por qué?**
- ¿**Cuántos** modelos uso?
- ¿**Cómo** los mezclo?
- ¿**Por qué mezclar modelos** puede dar **mejor que usar un solo modelo?**

Primer Ensamble – Majority Voting

Sin importar cuáles son los modelos subyacentes. Una posible idea es hacer varios modelos por separado y luego “juntarlos” de alguna manera.

- En clasificación: una posible opción es simplemente que cada modelo indique una clase y luego tomar la moda.
- En regresión: una posible opción es que cada modelo indique un valor de predicción y luego se tome la media (o la mediana o algún indicador similar).

Majority Voting

Imaginemos que tenemos:

- Un problema de clasificación binaria (clases A y B).
- 3 clasificadores que predicen la clase A con accuracy de 0.8.
- Una muestra en el conjunto de test de la clase A.

Si usamos los modelos por separado, tenemos un 0.2 de chances de etiquetar mal la muestra.

Si usamos un ensemble majority voting de los 3 modelos y **asumimos que los errores de los modelos están completamente decorrelacionados**, ¿cuáles son las chances de errarle?

Kappa de Cohen

La Kappa de Cohen es una métrica que analiza cuánto coinciden dos clasificadores, tomando en cuenta que podrían haber coincidido por casualidad.

Un valor alto de Kappa indican que los modelos se comportan parecido (lo cual no significa que anden ni bien ni mal).

$$k = \frac{p_o - p_e}{1 - p_e}$$

Cherrypicking

Cherrypicking es un término muy escuchado en Machine Learning para decir “agarraste exactamente lo que te conviene”

En ese código se vió que sirve para esos valores de semillas, pero ¿sirve en general?



Decorrelacionando

La clave de ensamblar diferentes modelos va a estar en lo decorrelacionados que están.

Podemos pensarlo de diferentes maneras:

- Desde un punto de vista de la varianza: la varianza del promedio de observaciones independientes con igual varianza es igual a dividir la varianza por la cantidad de observaciones. O sea, la varianza baja.
- Desde un punto de vista de los errores: si varios modelos tienen errores completamente decorrelacionados para una muestra, el promedio de esos errores tiende a desaparecer.

Bagging

¿Cómo podemos decorrelacionar los errores?

Si siempre utilizamos todo el conjunto de train, nuestros modelos estarán trabajando con la misma información. Además, siempre los mismos errores

Una primera idea entonces es entrenar varios modelos pero ocultando ciertas partes del conjunto de train, para que cada modelo trabaje con un conjunto distinto.

Para esto, utilizaremos el concepto de bootstrap.

Bootstrap

Bootstrap o *Bootstrapping* es una técnica de *resampling* para crear múltiples conjuntos nuevos de datos a partir de un dataset original. Para esto, la técnica va extrayendo datos (con reemplazo) al azar del dataset.

- Tomamos nuestro dataset original (train) de n muestras.
- Creamos un nuevo dataset vacío.
- Tomamos una muestra al azar del dataset original y lo ponemos en el nuevo dataset. Adicionalmente, la muestra es devuelta al dataset original (no se borra).
- Repetimos el paso anterior n veces, hasta obtener un dataset completo.
- Repetimos los pasos anteriores tantas veces como *dataset bootstraps* queramos tener.

Bagging

Mediante *bootstrap* podremos entonces generar diferentes datasets a partir del train original, y luego los podremos usar para entrenar diferentes modelos.

Bagging (*Bootstrap aggregating*) es un modelo de machine learning en donde ensamblamos los diferentes modelos entrenados sobre conjuntos de bootstrap del dataset original.

En resumen, tomamos el train original, creamos muchos conjuntos nuevos, los entrenamos y finalmente los *juntamos* de alguna forma útil.

Bagging - Pseudocódigo

Algorithm 1 Bagging

- 1: Let n be the number of bootstrap samples
 - 2:
 - 3: **for** $i=1$ to n **do**
 - 4: Draw bootstrap sample of size m , $\mathcal{D}_i$
 - 5: Train base classifier h_i on $\mathcal{D}_i$
 - 6: $\hat{y} = \text{mode}\{h_1(\mathbf{x}), \dots, h_n(\mathbf{x})\}$
-

En el caso de **regresión** podemos **tomar el promedio de las predicciones**.

Bagging - Out-of-Bag



Cuando hacemos bootstrap, muchas muestras quedan afuera del bootstrap data set. Las podemos usar como muestras de validación temporales.

Bagging – Decision Boundaries

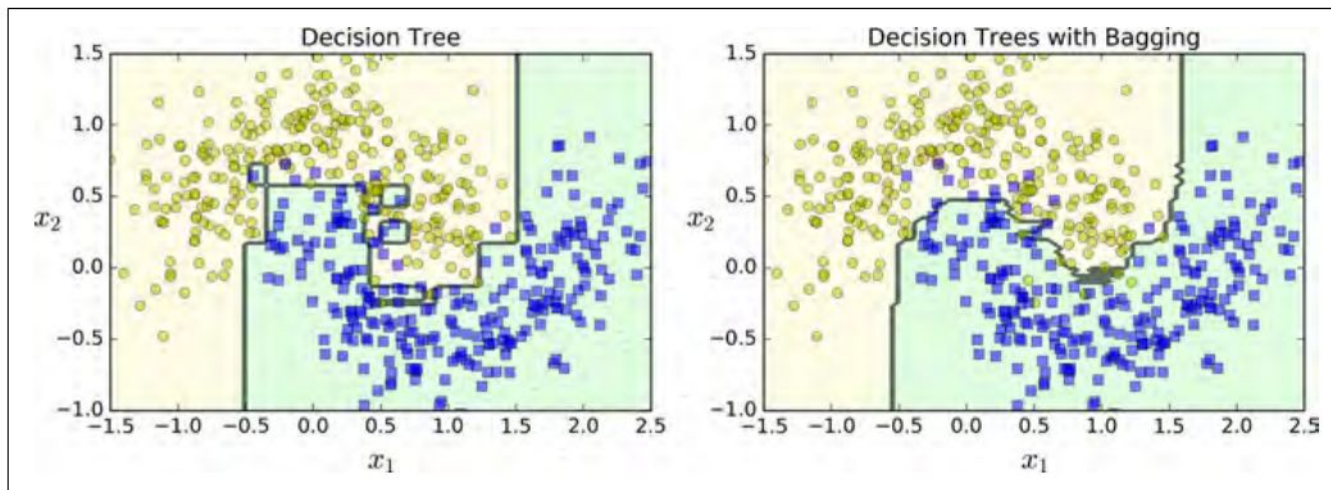


Figure 7-5. A single Decision Tree versus a bagging ensemble of 500 trees

Usando bagging podemos recrear fronteras que no sean paralelas a los ejes.

Bagging

Por **definición**, el concepto de Bagging **no está asociado a árboles de decisión**, aunque **funcionan muy bien como clasificador base**.

La idea interesante detrás del éxito de bagging viene de la mano de la búsqueda de la independencia en los errores de predicción.

Sorprendentemente **es preferible tener predictores “malos”, pero bien decorrelacionados**, a tener **predictores buenos pero con mucha correlación**.

Ya **decorrelacionamos los predictores sampleando las filas** ¿Qué podemos hacer para **decorrelacionar aún más los predictores**?

Random Forest

Ya sampleamos filas, ahora vamos por las columnas!

Las columnas que se eligen en los diferentes DecisionTree en un modelo de bagging no tienen por qué ser las mismas. ¿Por qué?

Sin embargo, podríamos forzar nosotros mismos a que diferentes predictores utilicen directamente un subconjunto distinto de las columnas para que sean predictores bien distintos.

En Random Forest vamos a tener un parámetro para controlar la cantidad de columnas que se eligen en cada árbol del bosque.

Random Forest – Cantidad de árboles

https://scikit-learn.org/stable/auto_examples/ensemble/plot_ensemble_oob.html

Los RandomForest son *bastante inmunes* al overfitting. La cantidad de árboles no es un parámetro que afecte al overfitting.

